Maintenance Operations Plan

Author: Justin Hunter
Date: 2026-04-14
Audience: Engineering team, operations team, and engineering leadership
Companion documents: Program Charter, Query Store Triage and Refactor Program plan, Scalability Roadmap

---

1. Purpose

This document is the operations side of the program. It covers the current state of nightly maintenance, the bugs and structural issues the MFC DBA team and I have found in the maintenance procedures, the recommendations that have been published to the iA team for review and deployment, the recommendations that are queued in the backlog, and the longer-arc move from iterative block deletes to partition switch operations.

The same multi-team structure that governs the procedure refactor program governs the maintenance work. The MFC DBA team and I produce the assessments, the corrective recommendations, and the supporting change proposals. The iA DBA team and the iA software development teams own the review, validation, non-production testing, and production deployment. A "fix" in this document means "a recommended change that has been published to the iA team," not "a change that has been deployed." Where a recommendation has been deployed, the change journal makes that explicit.

The detailed evidence behind this plan sits in two earlier assessment documents that the MFC DBA team and I have already produced:

- The nightly maintenance performance assessment (February 25, 2026). The end-to-end assessment of the orchestrating nightly maintenance procedure and the nineteen child procedures it calls.
- The purge optimization analysis (March 18, 2026). The set of code-level optimizations that can be applied now, before any partitioning work lands.

Both documents are available on request. The change journal that records what has been published as a recommendation, and which of those recommendations have been deployed, is maintained alongside the maintenance procedure source.

---

2. Current State

The nightly maintenance suite has grown organically over twenty-six versions. It handles database integrity checks, backups, index rebuilds, and a large set of data purge operations across more than thirty tables. The code base is defensive and structurally sound at the outer level. The orchestrating procedure manages the children predictably and traps errors. Several of the child procedures, however, contain issues that vary in severity from "silently does nothing" to "becomes uncomfortably slow at scale."

Three patterns drive the bulk of the issues.

The first pattern is bugs. A small number of the older child procedures have outright defects that were not caught at deployment time and have been latent in production for some time. The SmartShelf delete loops are the most consequential example: a `>` should be `<`, the loop never executes, and old SmartShelf errors and events are never being purged. Procedures with this kind of defect appear to be working from the outside (they run, they return without error, the table still exists) but are effectively no-ops.

The second pattern is unbounded loops. Several of the older procedures lack the `@MaxToDelete` safety parameter that the newer procedures have. They will loop until every qualifying row is gone, regardless of transaction log impact. On a first activation against a large backlog of qualifying rows, this can inflate the log enormously, occasionally to the point of filling the log drive.

The third pattern is iterative block deletion overhead at scale. This is the structural issue that drives the longer arc of the plan. Every delete procedure follows the same fundamental pattern: a `WHILE EXISTS` loop that deletes N rows per iteration. The pattern is sound at modest scale. It becomes expensive when the table being purged is in the hundreds of millions of rows. The expense is not the deletes themselves. It is the per-iteration overhead: the `EXISTS` check at the top of the loop, the temp table drop and recreate, and the consequent recompiles of the downstream statements. At a thousand iterations per run, the per-iteration overhead dominates the actual delete work.

---

3. Critical Bugs and What They Mean

The assessment surfaced several bugs. The two most consequential are reproduced here for visibility.

3.1 SmartShelf Delete Loops Never Execute

Procedures: `SmartShelf.lsp_DbDeleteOldSmartShelfErrors` and `SmartShelf.lsp_DbDeleteOldSmartShelfEvents`.

The `WHILE` condition reads:

```
While Exists(Select * From SmartShelf.EvtErrorLog Where DtTm < @CutoffDtTm)
  And @NumOfRowsDeleted > @MaxToDelete
```

`@NumOfRowsDeleted` is initialized to zero and `@MaxToDelete` defaults to twenty five million. The condition `0 > 25000000` is `FALSE` on the first evaluation, the loop body never executes, and the procedure exits without doing any work.

Effect: old SmartShelf errors and events are not being purged anywhere in the fleet. The tables grow without bound until manually pruned or until disk pressure forces operational attention.

Recommended change: flip the comparison from `>` to `<`, matching every other delete procedure in the suite. The change is one character per procedure. The recommendation is in the backlog awaiting iA review. The deployment is conditional on operations choosing the right cutover window, because the first activation against the backlog will move a non-trivial amount of data and the transaction log behavior should be observed before rolling the change to additional sites.

3.2 Double-Delete in `lsp_DbDeleteOldDiagEvents` and `lsp_DbDeleteOldErrors`

Both procedures issue two delete statements within each loop iteration, the second one using a self join into a `Top 50000` subquery. The second statement appears to be the original; the first appears to be an incomplete refactor. Both run in the same explicit transaction. Each iteration deletes up to one hundred thousand rows in a single transaction rather than the intended fifty thousand, and `lsp_DbDeleteOldErrors` lacks `@MaxToDelete` entirely.

Effect: wasted I/O, longer-than-intended transactions, and on first activation a real risk of transaction log inflation.

Recommended change: remove one of the two delete statements (the unconditioned `DELETE TOP (50000)` version is the right one to keep), drop the explicit `BEGIN TRAN / COMMIT TRAN` (each `DELETE` is already atomic), and add `@MaxToDelete` and `@NumOfRowsBlockSize` parameters consistent with the modern procedures. The recommendation is in the backlog awaiting iA review.

3.3 Other Items

The assessment lists several other items at varying severity. They are itemized in the source assessment and are not reproduced here in detail. They include:

- Inconsistent parameter conventions across the procedures, which makes the orchestration harder to reason about than it needs to be.
- Mixed defensive practice on temp table cleanup. Some procedures use the legacy `If Object_Id('TempDb..#X') Is Not Null Drop Table #X` pattern; the modern equivalent `Drop Table If Exists #X` is preferable.
- Several procedures that read the full row from a base table when only the row locator (the clustering key) is needed for the delete. This was the issue addressed in the v1-to-v2 change on `lsp_DbDeleteOldInvAuditEvents` recorded in the change journal.

---

4. Low-Hanging Fruit Already in the Plan

The purge optimization analysis enumerates the code-level optimizations that can be applied now, before partitioning. The four highest-impact items are summarized here.

Replace `WHILE EXISTS(SELECT *)` loop control with `@@ROWCOUNT` based loop control. Currently, every iteration of the main delete loop begins with a full `EXISTS` subquery against the base table. On a five hundred million row table being purged at a hundred thousand rows per block, that is five thousand unnecessary index seeks just to ask whether there is more work to do. The information is already available from the previous delete's `@@ROWCOUNT`. Switching to `@@ROWCOUNT` based control eliminates one full seek per iteration. Affects the main purge driver and the events purge.

Stop dropping and recreating the per-block temp table every iteration. The current pattern drops the temp table, creates a new one via `SELECT INTO`, and creates a new index on it on every loop pass. The temp table itself is small, so the index build is cheap, but every downstream statement that joins to the temp table gets a fresh plan compilation on every iteration. With approximately fifteen downstream `DELETE` statements per iteration, the compile overhead on a multi-thousand-iteration run is substantial. The modern pattern (create the temp table once before the loop, truncate and refill it inside the loop) keeps the same plans cached across iterations.

Narrow the block selector reads. Several block selectors do `SELECT TOP (N) * FROM ... ORDER BY Id` when only `Id` is needed. The fix changes `*` to the specific column list, which lets the query be satisfied from the clustered index alone. This pattern was already applied to `lsp_DbDeleteOldInvAuditEvents` in the v1-to-v2 change. The same change should be applied to the remaining purge procedures that follow the same shape.

Use `@@ROWCOUNT` for accurate row count tracking. Several procedures increment `@NumOfRowsDeleted` by `@NumOfRowsBlockSize` rather than by `@@ROWCOUNT`. The value drifts on the final iteration where the actual deleted count is smaller than the block size. Switching to `@@ROWCOUNT` makes the tracking exact and the `@MaxToDelete` exit condition precise.

The full list with rationale for each item is in the purge optimization analysis.

---

5. The Long Arc: Partition Switch Operations

The structural issue with iterative block deletion is that the work scales with the number of rows being deleted, even when the rows being deleted represent a contiguous time range that is logically a single chunk. Deleting a hundred million rows from a five hundred million row table over the course of a maintenance window is a fundamentally different operation than swapping out a partition of the table that already contains those rows.

The partitioning effort (the five-module POC summarized in the Scalability Roadmap) is the architectural enabler that lets the maintenance plan move from iterative deletes to partition switches on the highest-volume tables. The flow is straightforward once the table is partitioned: at the end of the retention window, the oldest partition is switched out into a staging table, the staging table is truncated, and the now-empty partition becomes the next month's (or week's) write target. The whole operation is metadata-only at the partition level, completes in seconds rather than hours, and produces almost no transaction log activity.

The maintenance procedures will need to change to support this. Specifically:

- The high-volume purge procedures (`lsp_DbDeleteOldEvents` , `lsp_DbDeleteOldRxData` , `lsp_DbDeleteOldDocumentImages` , `lsp_DbDeleteOldScriptImages` , and the rest of the procedures that purge against the partitioning candidate tables) will gain a partition-aware code path that prefers `SWITCH PARTITION` to iterative deletes when the target table is partitioned.
- A new procedure or set of procedures will manage the partition lifecycle (sliding window: split forward, switch out the oldest, merge as needed). The POC modules already provide the building blocks. Productionizing them is a Phase 2 deliverable.
- The orchestrating nightly procedure will need to know which tables are partitioned and route to the right purge procedure, or the purge procedures will detect the partitioning state internally and choose the right path.

The expected outcome on a fully partitioned high-volume table is that the per-table purge window collapses from hours to seconds. The total nightly maintenance window contracts proportionally. The transaction log activity associated with retention drops to near zero. None of

these outcomes are speculative. They are the standard properties of partition switching as documented in the POC modules and as observed in the literature.

---

6. Maintenance Schedule and Observability Recommendations

Several recommendations sit at the operations layer rather than the code layer. They do not require schema changes; they require operational discipline and a small amount of tooling.

Confirm Query Store is enabled and configured at every site. The diagnostic toolkit assumes Query Store is on. Where it is not, we lose the ability to do plan stability analysis and to track outcomes against baseline. This is a Phase 1 housekeeping item that operations can confirm in an afternoon.

Capture the Query Store top fifty per site on a quarterly cadence. The May 7, 2026 capture is the current baseline. The next capture is due in early August 2026. The capture is the same script run at every site. The output is an Excel workbook with one tab per MFC. Cadence beats precision. Even a slightly dated capture is better than no capture.

Capture data velocity and capacity headroom on the same cadence. The data velocity and capacity script in the diagnostic toolkit produces per-table growth rates per site. The output should be captured alongside the Query Store roll-up so that capacity decisions and partition retention decisions are informed by current data. Linear extrapolation will undercount seasonal growth, which is fine for a leading indicator but should not be the sole input to a long-horizon capacity decision.

Watch the nightly maintenance run time per procedure. The current logging captures elapsed time per child procedure. That data should be aggregated per site per week and trended. A purge procedure whose elapsed time is growing faster than the table is growing is a signal that the procedure is starting to feel the iterative loop overhead. Procedures whose elapsed time is flat against table growth are healthy.

Confirm the SmartShelf bug fix landed before the next quarterly review. Once the fix is deployed, the affected tables will start to drain. The drain may be sudden if the backlog is large. Operations should know this is coming and be prepared for the transaction log volume that the drain produces.

Stagger the bug fix deployments across sites. The SmartShelf fix and the double-delete cleanup will both move significant data on first activation. Rolling them out one site at a time, with a window of observation between sites, lets us catch any transaction log surprise before it lands at every site simultaneously.

Monitor per-site behavior of every deployed maintenance change. The same post-deployment monitoring discipline that governs the procedure refactor program applies here. After a maintenance change reaches production at a site, the MFC DBA team and I capture the elapsed time of the affected child procedure, the row count it deleted, and the transaction log activity it generated, and we compare those against the pre-deployment baseline at the same site. The data shape and the purge backlog at each site are different. A change that runs cleanly at the first site will not necessarily run cleanly at every site. Where a site shows unexpected behavior after a maintenance change, we open a per-site root cause analysis on the same template the procedure refactor program uses (data shape, plan choice, statistics quality, parameter selection, schema variance) and document the outcome.

---

7. Phasing

The maintenance work runs in three waves that line up roughly with the phases in the program charter. The waves describe the cataloging and recommendation work that the MFC DBA team and I produce. Production deployment of any specific recommendation runs on the iA team's cadence and is not committed by these waves.

Wave 1: Bug-fix recommendations and the highest-impact code optimizations. This wave is happening now. The SmartShelf bug-fix recommendation, the double-delete cleanup in `lsp_DbDeleteOldDiagEvents` and `lsp_DbDeleteOldErrors`, the `@MaxToDelete` retrofit on the older procedures, the `WHILE EXISTS` to `@@ROWCOUNT` conversion, the temp table reuse pattern, and the wide column read narrowing all fall in this wave. Each is a small, scoped, individually reviewable recommendation with a corresponding entry in the change journal.

Wave 2: Modernization at scale. Once the Wave 1 recommendations are accepted and patterns are established, the focus shifts to publishing the same code-level patterns consistently across all of the maintenance procedures. This is mostly mechanical work informed by the patterns from Wave 1. It also includes the legacy `Object_Id` to `Drop Table If Exists` substitution and the parameter convention alignment across the procedures.

Wave 3: Partition switch integration. Wave 3 begins once the partitioning rollout (per the Scalability Roadmap) has produced its first partitioned production table. The first partition-aware purge procedure recommendation goes to the iA team for that table. The remaining high-volume procedure recommendations follow as their tables become partitioned. The orchestrator change recommendation to detect partitioning state and route appropriately also lands in this wave.

---

8. Expected Outcomes

The maintenance plan is not where the largest single read reduction lives. That is the refactor program. The maintenance plan delivers two different outcomes.

The first outcome is operational reliability. The bug fixes restore purges that are not currently running. The `@MaxToDelete` retrofits eliminate the risk of transaction log inflation on first activation. The temp table reuse and `@@ROWCOUNT` based loop control reduce the per-iteration overhead that has been making the maintenance window slowly grow. Operations should see fewer maintenance window overruns and fewer transaction log incidents.

The second outcome is graceful scaling. With partition switches in place on the highest-volume tables, the maintenance window for those tables stops being a function of how many rows are being purged and becomes a function of how many partitions are being swapped. Adding twenty percent more data to a table does not lengthen the purge window for that table. Adding two more sites does not slow the maintenance window at the existing sites. The architecture stops compounding its own difficulty.

The two outcomes together are the maintenance contribution to the program goal of supporting two to three times the current data volume on the same hardware with steady or improving response times.

---

9. Change Management and Reporting

Each maintenance change recommendation flows through the same multi-stage pipeline that governs the procedure refactor program: review by the iA DBA team, validation by the iA software development team responsible for the affected functional area, non-production testing, and production deployment. Recommendations are published into the change journal with the same structure used for the existing entries: procedure, affected table, summary, before-and-after code excerpts, and rationale. The journal also carries the deployment status of each recommendation (Cataloged, In iA Review, In Non-Prod Testing, In Production), so that the gap between recommended-and-ready and deployed-to-production is visible at a glance.

I will report against the maintenance plan as part of the program-level weekly review. The reporting carries three numbers: the recommendation pace (count of new entries published in the journal over a window), the deployment pace (count of those entries that have moved to production), and the post-deployment per-site outcome distribution (count of sites closed as Improved, Unchanged, or Regressed for the maintenance changes that have reached production). Significant operational events (transaction log incidents, maintenance window overruns, purges that produced unexpected row counts) get logged against the relevant procedure in the journal so that the operational history of each procedure is preserved.

By the end of Phase 3, the journal is the historical record of how the maintenance suite was modernized, and it is the document a future engineer will read when they need to understand why a procedure looks the way it does today.